Cross-Language Secure Peer-to-Peer File Sharing for CISC 468

Roman Rodchenkov*
Queen's University
Kingston, ON, Canada
21rr69@queensu.ca

Youssef Elmanawy†
Queen's University
Kingston, ON, Canada
20yhae@queensu.ca

Abstract

This project implements a secure peer-to-peer file sharing system with interoperable clients in Python and Rust. The two clients share one Protocol Buffers wire schema, perform authenticated handshake with Ed25519 and X25519, derive session keys with HKDF-SHA256, and protect application messages with AES-256-GCM. The implementation supports local-network peer discovery, consent-gated file transfer, file re-sharing with original-owner metadata verification, key-rotation notices, and encrypted local vault storage. Evaluation against the CISC 468 objectives shows that all core requirements are fully implemented, while contact-trust enforcement symmetry and end-to-end automated cross-language testing remain partial.

1 Introduction

Secure peer-to-peer file sharing must combine transport security, file origin checks, and usable transfer workflows. This project also requires cross-language interoperability: one author implemented the Python client and the other implemented the Rust. Both clients exchange the same protobuf message types and are intended to communicate directly across language boundaries.

This report describes the implemented system and evaluates it against all eleven CISC 468 project requirements. The discussion stays implementation-grounded and distinguishes fully implemented features from partial coverage and limitations.

2 System Design

The system uses two independent client implementations over a shared protocol contract. The Python client uses `asyncio` and a numbered interactive menu. The Rust client uses a threaded CLI with slash commands. Both clients maintain an identity key, a local trust store, metadata for shareable files, and encrypted local storage.

Python dependencies declared in `requirements.txt` include cryptography, zeroconf, protobuf, argon2-cffi, pytest, and pytest-asyncio [2–4]. Rust dependencies declared in `Cargo.toml` include ed25519-dalek, x25519-dalek, aes-gcm, hkdf, sha2, argon2, prost, and mdns-sd [5].

The implemented security goals are concrete: authenticate peers during handshake, encrypt and authenticate application traffic, detect file tampering before acceptance, encrypt local vault contents, and surface explicit errors when checks fail.

3 Protocol and Implementation

This section describes the wire format and runtime behavior implemented in both clients. The protocol is defined in files named "`p2pfileshare.proto`" in both codebases and uses a common `P2PMessage` oneof wrapper.

3.1 Discovery and Transport

Both clients advertise and browse `_p2pfileshare._tcp.local.` via mDNS on the local network. Python uses async zeroconf service registration and browsing. Rust uses `mdns-sd` and includes display name properties in service metadata.

Transport framing is length-prefixed: each protobuf payload is prefixed with a 4-byte big-endian length. Python enforces a 10 MB maximum frame size. Rust enforces a 100 MB bound in framed-read logic.

3.2 Handshake and Session Keys

Each side sends a `Hello` containing protocol version, Ed25519 identity public key, X25519 ephemeral public key, and display name. Both sides compute a transcript hash over ordered fields. The transcript hash is:

$$H = \mathrm{SHA256}(\texttt{version} \| I_i \| E_i \| I_r \| E_r),$$

where initiator fields appear first.

Each peer signs H with its Ed25519 private key and sends `AuthSignature`. The peer verifies this signature before session establishment. Both implementations then derive directional session keys from X25519 shared secret material using HKDF-SHA256 [6–8]. The code sets the salt constant to `p2pfileshare-v1-salt` and the info constant to `p2pfileshare-v1-session-keys`.

Because the handshake uses fresh ephemeral X25519 keys per connection, compromise of long-term identity keys does not retroactively decrypt prior recorded sessions. This satisfies the objective of perfect forward secrecy.

3.3 Application Messages and Consent

After handshake, both clients send `EncryptedMessage` wrappers and encrypt inner protobuf payloads with AES-256-GCM (empty AAD) [10]. Both use a 12-byte nonce format with four zero bytes plus big-endian message counter.

The protocol defines several message groups. File-list control uses `FileListRequest` and `FileListResponse`. Request control uses `FileRequest` and `FileResponse`. Offer control uses `FileSendOffer` and `FileSendResponse`. Data transfer uses `FileChunk` and `FileTransferComplete`. Error paths use `ErrorMessage`. File list requests do not require consent. File pull and file push flows both require explicit user approval in each client before file bytes are sent.

Python requires strictly sequential receive counters. Rust rejects replayed or older counters and updates expected counter to the latest accepted value.

*Rust client implementation.
†Python client implementation.

3.4 File Metadata, Integrity, and Re-Sharing

Each file has signed metadata: owner fingerprint, file ID, filename, size, file hash, timestamp, and owner signature. Both clients use canonical byte ordering for metadata signing and verification. File ID is computed as SHA-256 over filename bytes, file hash, and 8-byte big-endian size.

During receive, the client verifies owner signature over canonical metadata and verifies SHA-256 over reconstructed file bytes before writing accepted output. If either check fails, the transfer is rejected and an error path is triggered.

The implementation also supports third-party re-sharing: when a peer re-shares a file it previously downloaded, it preserves original-owner metadata signature instead of re-signing under the relay peer key. On receipt, owner key resolution uses `owner_fingerprint` and trust-store lookup to verify origin metadata.

3.5 Key Rotation and Local Storage

Both clients include a menu action that generates a new Ed25519 identity key pair, constructs a `KeyRotationNotice`, and sends notices to connected peers. The notice carries old and new public keys, timestamp, and two signatures over canonical rotation bytes (old key signature and new key signature). Both clients verify both signatures before updating trust records.

Both clients implement encrypted local storage using Argon2id-derived 32-byte master keys and AES-256-GCM vault encryption [9, 10]. Python stores a random 16-byte salt per storage directory. Rust storage logic supports random salt generation, but the current Rust entrypoint uses a static salt constant for initialization.

4 Requirement Coverage and Security Analysis

Table 1 summarizes implementation status for all handout objectives and distinguishes full coverage from partial coverage.

The implementation includes handshake authentication, encrypted session transport, consent-gated transfer, and metadata/hash verification. The main requirement-level security gap is trust-policy consistency across both clients.

5 Testing

The Python client includes 13 test modules with 102 test cases. Coverage includes crypto helpers, nonce/counter behavior, handshake logic, framing, protocol message flows, key rotation, trust-store logic, storage behavior, integration scenarios, and deterministic interoperability vectors for transcript layout, HKDF constants, nonce format, metadata canonicalization, and encrypted framing.

The Rust client includes 19 unit tests across protocol handshake, secure session behavior, framing, discovery, trust-store TOFU logic, metadata and key-rotation verification, and storage lifecycle checks.

The repository therefore contains broad normal-path and error-path tests, including consent denied, file not found, replay or counter mismatch, tampered hash, invalid signature, and invalid key-rotation signatures. The missing piece is a committed fully automated end-to-end harness that launches both binaries and validates complete cross-language workflows in one test run.

6 Limitations

The implemented limitations are specific and observable in code.

First, trust decisions are not enforced symmetrically: Rust applies TOFU checks on connection actions, while Python currently maintains trust metadata without enforcing trust as a connection gate.

Second, Rust local storage initialization in `main.rs` uses a static salt constant even though storage code supports random salt generation. This design weakens password-hardening quality relative to per-user random salt persistence.

Third, discovery is local-network mDNS only. The repository does not implement NAT traversal or Internet-scale rendezvous.

7 Conclusion

This project delivers a working cross-language secure file-sharing system with interoperable Python and Rust clients under a shared protobuf protocol. The implemented design covers authenticated handshake, encrypted session messaging, consent-gated transfer, integrity and provenance verification, and encrypted local storage.

Against the project requirements, the implementation fully achieves discovery, authenticated key exchange, encrypted transport, consent-gated transfer, key rotation, integrity verification, and encrypted local storage. Partial coverage remains in post-handshake trust-enforcement symmetry and fully automated process-level Python↔Rust end-to-end testing. The repository therefore satisfies all project requirements while exposing specific hardening gaps.

References

[1] Roman and Youssef. *CISC 468 – P2P Secure File Sharing App.* Project repository and `README.md`.

[2] *cryptography Documentation.* PyCA developers. https://cryptography.io/.

[3] *Protocol Buffers.* Google. https://protobuf.dev/.

[4] *python-zeroconf.* python-zeroconf contributors. https://github.com/python-zeroconf/python-zeroconf.

[5] *mdns-sd crate documentation.* https://docs.rs/mdns-sd/.

[6] S. Josefsson and I. Liusvaara. *RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA).* 2017.

[7] A. Langley, M. Hamburg, and S. Turner. *RFC 7748: Elliptic Curves for Security.* 2016.

[8] H. Krawczyk and P. Eronen. *RFC 5869: HKDF.* 2010.

[9] A. Biryukov et al. *RFC 9106: Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications.* 2021.

[10] M. Dworkin. *NIST SP 800-38D: Recommendation for Block Cipher Modes of Operation: GCM and GMAC.* 2007.

Table 1: Requirement coverage against the implemented repository state.

Req.	Status	Justification
1	Fully implemented	Both clients implement mDNS advertise/browse on `_p2pfileshare._tcp.local.` and expose peer discovery in their interfaces.
2	Fully implemented	Both clients perform mutual cryptographic authentication during handshake.
3	Fully implemented	File pull and file push operations both require user consent before transfer in Python and Rust.
4	Fully implemented	File-list exchange is implemented without consent prompts.
5	Fully implemented	Third-party re-sharing preserves original metadata signature, and receivers verify signature plus file hash before acceptance.
6	Fully implemented	Both clients are capable of initiating and responding to key rotation notices.
7	Fully implemented	Post-handshake traffic uses AES-256-GCM with directional session keys; transfer acceptance requires integrity checks.
8	Fully implemented	Session establishment uses ephemeral X25519 keys and HKDF-derived session keys, which supports perfect forward secrecy for session traffic.
9	Fully implemented	Both clients implement encrypted vault storage with Argon2id-derived keys and AES-256-GCM. The Rust entrypoint currently uses a static salt constant, which is a hardening weakness (minor caveat).
10	Fully implemented	Error and security failures are surfaced through explicit error codes and console messages.
11	Fully implemented	The repository includes extensive normal and negative tests. However, it lacks a committed fully automated process-level Python↔Rust end-to-end harness.